

signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

git 0f0fdea → 2026-08-03 · 0fb5559+수정중 | 5개 파일 · +155 / -305 줄 · 코드 29쪽 | 새 코드북 70쪽

바뀐 내용

- 실신호 결과를 손으로 받아쳐 넘기는 한 줄이 생겼습니다 — analyze/survey 에 --brief 를 붙이면 맨 끝에 sig2 로 시작하는 요약 한 줄과 4글자 오타 검출 코드가 붙습니다.
- 파일명 읽기를 고쳤습니다 — 새 점(.) 형식 이름을 CLI 와 웹이 같은 규칙으로 읽고, 이름에 섞인 라벨 토막(u8·real 따위)이 진짜 제원을 덮어쓰던 오독을 막았습니다.
- 합성 신호 생성기와 채점은 인쇄물에서 뺐습니다 — gen.py 는 필사본에서 지우고, cli.py 를 새로 쓰면 gen/dataset/sweep 명령이 사라집니다. 그 코드는 앞으로 개발기에서만 씁니다.
- pyproject 는 개발기용 발급기 의존성 한 줄이 늘었을 뿐, 장비 동작과는 무관합니다.

□ pyproject.toml	+2		2쪽	코드북 2쪽
□ signus/sigio.py	+44	-16	2쪽	코드북 4쪽
□ signus/pipeline.py	+4	-2	5쪽	코드북 35쪽
□ signus/cli.py	+74	-276	14쪽	코드북 44쪽
□ signus/web/app.js	+31	-11	18쪽	코드북 59쪽
□ signus/gen.py — 삭제됨 (필사본에서 지운다)				

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · **붉은 점선** = 그 자리에서 몇 줄 사라짐 · **↳** = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

pyproject.toml +2 · 코드북 2쪽 · 새로 쓸 줄 14-15

```

1 1  [build-system]
2 2  requires = ["setuptools>=68"]
3 3  build-backend = "setuptools.build_meta"
4 4
5 5  [project]
6 6  name = "signus"
7 7  version = "2.0.0"
8 8  description = "Blind PSK/QAM demodulator: auto parameter detection + constellation UI"
9 9  requires-python = ">=3.11"
10 10 dependencies = ["numpy>=1.26", "scipy>=1.11"]
11 11
12 12 [project.optional-dependencies]
13 13 dev = ["pytest>=8.0", "ruff>=0.4"]
14 + # 필사 인쇄물 발급기(tools/codebook.py) 전용. 분석기 본체는 여전히 numpy+scipy 둘뿐이다.
15 + tools = ["pygments>=2.17"]
16 16 # Optional JIT for the sequential equalizer/carrier loops (16-34x on those loops).
17 17 # Falls back to identical pure-numpy when absent. NOTE: numba pins numpy<2.5.
18 18 fast = ["numba>=0.60"]
19 19
20 20 [project.scripts]
21 21 signus = "signus.cli:main"
22 22
23 23 [tool.setuptools.packages.find]
24 24 include = ["signus*"]
25 25
26 26 [tool.setuptools.package-data]
27 27 signus = ["web/*"] # the served frontend: without this a pip install 404s the whole UI
28 28
29 29 [tool.pytest.ini_options]
30 30 pythonpath = ["."]
31 31 testpaths = ["tests"]
32 32 addopts = "-q"
33 33 markers = ["stretch: slow/marginal cases, opt-in"]
34 34
35 35 [tool.ruff]
36 36 target-version = "py311"
37 37 line-length = 100
38 38
39 39 [tool.ruff.lint]
40 40 select = ["E", "F", "I", "UP", "B", "SIM", "NPY"]

```

signus/sigio.py +44 -16 · 코드북 4쪽 · 새로 쓸 줄 1, 12-15, 26-27, 49-69, 71-75, 77, 79-83, 111-115

```

1 + """Sample file I/O. Filename carries read-necessities ONLY (fs, cplx|real, sample
2 2 type, endian, bitrev); ground truth lives in a `<filename>.json` sidecar the
3 3 analyzer never reads. SigMF sidecars (`<stem>.sigmf-meta`) are also understood."""
4 4
5 5 import json
6 6 import os
7 7 import re
8 8 from dataclasses import dataclass
9 9
10 10 import numpy as np
11 11
12 + # fs/rf 토막은 옛 밀줄 형식 전용이다. 점으로도 끊길 수 있어 경계에 둘 다 넣는다 -- 새 형식은
13 + # 샘플레이트를 점두사 없이 cplx|real 바로 뒤에 놓으므로 이 정규식에 걸리지 않는다.
14 + _FS_TOK = re.compile(r"(?:^[. _])fs(\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[. _]|$)", re.I)
15 + _RF_TOK = re.compile(r"(?:^[. _])rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[. _]|$)", re.I)
16 16
17 17 # token -> (numpy code, full-scale divisor, offset). u-types are offset binary

```

```

16 18 # (e.g. 80 / RTL-SDR u8: 0..255 with 128 = zero).
17 19 _DTYPES = {
18 20     "i8": ("i1", 128.0, 0.0), "u8": ("u1", 128.0, 128.0),
19 21     "i16": ("i2", 32768.0, 0.0), "u16": ("u2", 32768.0, 32768.0),
20 22     "f32": ("f4", 1.0, 0.0), "f64": ("f8", 1.0, 0.0),
21 23 }
22 24 # baudline-style aliases: <bits>t = two's complement, <bits>o = offset binary
23 25 _ALIAS = {"8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64"}
26 + _TOK = {v: k for k, v in _ALIAS.items()} # 저장할 땐 baudline 표기로 되돌린다
27 + _FMT = {"cplx": "iq", "real": "real", "iq": "iq"} # iq = 옛 밑줄 형식의 이름
24 28 _BITREV = np.array([int(f"{i:08b}"[::-1]), 2) for i in range(256)], dtype=np.uint8)
25 29
26 30 # SigMF core:datatype -> (dtype token, fmt, endian)
27 31 _SIGMF = {"cf32": ("f32", "iq"), "ci16": ("i16", "iq"), "ci8": ("i8", "iq"),
28 32     "cu8": ("u8", "iq"), "rf32": ("f32", "real"), "ri16": ("i16", "real")}
29 33
30 34
31 35 @dataclass
32 36 class Meta:
33 37     fs: float | None = None
34 38     fmt: str | None = None # 'iq' | 'real'
35 39     dtype: str = "i16" # key of _DTYPES
36 40     endian: str = "le" # 'le' | 'be' (floats included)
37 41     bitrev: bool = False # bit order reversed within each byte
38 42     rf_center: float | None = None # RF centre freq (Hz); None = unknown (report baseband)
39 43
40 44     def ok(self) -> bool:
41 45         return self.fs is not None and self.fmt in ("iq", "real") and self.dtype in _DTYPES
42 46
43 47
44 48 def parse_name(name: str) -> Meta:
49 +     """파일명이 읽기 필수 정보를 실어 나른다 (정답은 사이드카에만 있고 절대 읽지 않는다).
50 +
51 +     새 형식 <이름>.cplx|real.<샘플레이트 정수>.<16t>[.be][.bitrev].pcm
52 +     옛 형식 <이름>_fs<샘플레이트>_iq|real_<i16>[_be][_bitrev].iq
53 +
54 +     샘플레이트는 새 형식에서 접두사 없이 온다 -- 그래서 '숫자처럼 생긴 토막'이 아니라
55 +     cplx|real **바로 다음 자리**로 찾는다. 이름 자체가 숫자여도(20260801.cplx...) 안 헛갈린다.
56 +     확장자를 떼지 않고 통째로 쪼갬다: 새 형식은 마지막 토막(.pcm)이 포맷을 뜻하지 않고,
57 +     옛 형식은 확장자(.iq)가 이미 다른 토막과 같은 말을 하므로 어느 쪽도 손해가 없다."""
58 +     base = os.path.basename(name).lower()
59 +     toks = re.split(r"[.]", base)
60 +     # 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 **마지막** 후보. 세 오독을 막는다
61 +     :
62 +     # · 라벨의 real/cplx/iq(rec_iq_20260801.cplx...)가 fs/fmt 를 강탈 -- 마지막 후보 규칙이 막고
63 +     ,
64 +     # · 점 낀 샘플레이트(cap.cplx.2.4e6... -> fs=2 Hz)와 라벨 숫자를 fs 로 발명(..._iq_20260728.ra
65 +     w)
66 +     # -- 숫자 뒤가 제원 토막(16t/be/.../pcm)이어야 한다는 관문이 막는다. 관문에 걸리면 fs 없음
67 +     으
68 +     # 크게 죽는다: 조용한 오답 대신 시끄러운 실패가 이 저장소의 원칙이다.
69 +     cands = [k for k, t in enumerate(toks) if t in _FMT]
70 +     hits = [k for k in cands if k + 2 < len(toks) and toks[k + 1].isdigit()
71 +         and (toks[k + 2] in _DTYPES or toks[k + 2] in _ALIAS
72 +         or toks[k + 2] in ("be", "bitrev", "pcm") or toks[k + 2].startswith("rf"))]
73 +     i = hits[-1] if hits else (cands[0] if cands else None)
74 +     m = Meta()
75 +     if i is not None: # 'is not None' 이어야 한다 -- 0 번 토막(cplx...)도 유효
76 +         하다
77 +         m.fmt = _FMT[toks[i]]

```

```

73 +         if i in hits:
74 +             m.fs = float(toks[i + 1])
75 +         if m.fs is None and (mt := _FS_TOK.search(base)):
49 76             m.fs = float(mt.group(1))
77 +         if rt := _RF_TOK.search(base):
51 78             m.rf_center = float(rt.group(1))
79 +         tail = toks[i + 1:] if i is not None else toks    # 제원은 포맷 토막 **뒤**에만 산다 --
80 +         m.dtype = next((_ALIAS.get(t, t) for t in tail    # 라벨의 u8/f32/be 가 제원을 오염 못 하게
81 +                        if t in _DTYPES or t in _ALIAS), "i16")
82 +         m.endian = "be" if "be" in tail else "le"
83 +         m.bitrev = "bitrev" in tail
57 84         return m
58 85
59 86
60 87 def parse_sigmf(path: str) -> Meta | None:
61 88     """Read `<stem>.sigmf-meta` (core:sample_rate + core:datatype) if present."""
62 89     try:
63 90         with open(os.path.splitext(path)[0] + ".sigmf-meta") as fh:
64 91             doc = json.load(fh)
65 92             g = doc["global"]
66 93             code = g["core:datatype"].replace("_le", "").replace("_be", "")
67 94             dtype, fmt = _SIGMF[code]
68 95             endian = "be" if g["core:datatype"].endswith("_be") else "le"
69 96             fs = float(g["core:sample_rate"])
70 97         except (OSError, KeyError, ValueError, TypeError, AttributeError):
71 98             return None # MANDATORY fields malformed: fall back to filename tokens
72 99         try:
73 100             # rf centre is OPTIONAL: a malformed value must cost only this field, never the whole
74 101             # sidecar -- discarding valid fs/fmt/dtype here silently re-read the file with filename
75 102             # tokens, which can lie about fs (a quiet garbage decode).
76 103             caps = doc.get("captures") or []
77 104             rf = caps[0].get("core:frequency") if caps else None
78 105             rf = None if rf is None else float(rf)
79 106         except (KeyError, ValueError, TypeError, AttributeError, IndexError):
80 107             rf = None
81 108         return Meta(fs, fmt, dtype, endian, rf_center=rf)
82 109
83 110
111 + def make_name(label: str, m: Meta) -> str:
112 +     """저장은 새 형식만: <이름>.cplx|real.<샘플레이트>.<16t>[.be][.bitrev].pcm"""
113 +     extra = ("be" if m.endian == "be" else "") + (".bitrev" if m.bitrev else "")
114 +     kind = "real" if m.fmt == "real" else "cplx"
115 +     return f"{label}.{kind}.{m.fs:.0f}.{_TOK.get(m.dtype, m.dtype)}{extra}.pcm"
88 116
89 117
90 118 def decode(raw: bytes, meta: Meta) -> np.ndarray:
91 119     """Raw bytes -> samples: complex128 for 'iq', float64 for 'real'.
92 120     Tolerates truncated files (drops trailing partial samples)."""
93 121     code, scale, off = _DTYPES[meta.dtype]
94 122     if meta.bitrev:
95 123         raw = _BITREV[np.frombuffer(raw, dtype=np.uint8)].tobytes()
96 124     step = np.dtype(code).itemsize
97 125     dt = ("<" if meta.endian == "le" else ">") + code
98 126     x = np.frombuffer(raw[: len(raw) - len(raw) % step], dtype=dt).astype(np.float64)
99 127     x = (x - off) / scale
100 128     if meta.fmt == "iq":
101 129         x = x[: x.size & ~1] # drop dangling half-sample
102 130         return x[0::2] + 1j * x[1::2]
103 131     return x
104 132

```

```

105 133
106 134 def read(path: str, meta: Meta | None = None) -> tuple[np.ndarray, Meta]:
107 135     meta = meta or parse_sigmf(path) or parse_name(path)
108 136     if not meta.ok():
109 137         raise ValueError(f"need fs + fmt (filename tokens, SigMF, or --fs/--fmt): {path}")
110 138     with open(path, "rb") as fh:
111 139         x = decode(fh.read(), meta)
112 140     if x.size < 256:
113 141         raise ValueError(f"신호가 너무 짧습니다 ({x.size} samples): {path}")
114 142     return x, meta
115 143
116 144
117 145 def write(path: str, x: np.ndarray, meta: Meta) -> None:
118 146     """Quantize (ints: scaled to 99.9th-percentile full scale) and write interleaved."""
119 147     code, scale, off = _DTYPES[meta.dtype]
120 148     flat = np.column_stack([x.real, x.imag]).ravel() if np.iscomplexobj(x) else x.real
121 149     if scale != 1.0: # integer types
122 150         peak = np.percentile(np.abs(flat), 99.9) or 1.0
123 151         flat = np.round(flat * (scale - 1) / peak) + off
124 152         info = np.iinfo(code)
125 153         flat = np.clip(flat, info.min, info.max)
126 154     out = flat.astype("<" if meta.endian == "le" else ">") + code)
127 155     if meta.bitrev:
128 156         out = np.frombuffer(_BITREV[np.frombuffer(out.tobytes(), np.uint8)].tobytes(),
129 157                             dtype=out.dtype)
130 158     out.tofile(path)
131 159
132 160
133 161 def sidecar_write(path: str, meta: Meta, truth: dict, gen: dict) -> None:
134 162     doc = {"fs": meta.fs, "fmt": meta.fmt, "dtype": meta.dtype,
135 163           "endian": meta.endian, "bitrev": meta.bitrev, "truth": truth, "gen": gen}
136 164     with open(path + ".json", "w") as fh:
137 165         json.dump(doc, fh, indent=1)
138 166
139 167
140 168 def sidecar_read(path: str) -> dict | None:
141 169     try:
142 170         with open(path + ".json") as fh:
143 171             return json.load(fh)
144 172     except (OSError, ValueError):
145 173         return None

```

signus/pipeline.py +4 -2 · 코드북 35쪽 · 새로 쓸 줄 104-106, 433

```

1 1  """End-to-end blind demodulation. Two families:
2 2     linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
3 3         classify -> fine sync -> [equalizer rescue] -> quality
4 4     fsk (CPFSK/MSK): frequency discriminator (gated by constant envelope)
5 5     Classification runs BEFORE fine sync so the decision-directed loop knows the
6 6     constellation. Differential demapping is an option, not a detection: dqpsk and
7 7     qpsk share a constellation."""
8 8
9 9     import json
10 10    from dataclasses import dataclass, field
11 11
12 12    import numpy as np
13 13
14 14    from . import classify as cl
15 15    from . import dsp, triage
16 16    from .channelize import extract
17 17    from .chirp import analyze_chirp, is_chirp, sweeps_band

```

```

18 18 from .constellations import demap_bits, demap_diff_bits, mod_order
19 19 from .detect import Detection, detect
20 20 from .eq import equalize, equalize_fse
21 21 from .fsk import analyze_fsk, fsk_gate
22 22 from .sigio import Meta, parse_name, read
23 23 from .spectrum import spectrum, waterfall
24 24 from .sync import find_preamble
25 25
26 26 _BITS_CAP = 65536
27 27 _EQ_LOCK = 60.0 # below this a linear signal is worth an equalizer attempt
28 28 _EQ_GAIN = 3.0 # ...keep it only if lock improves by this much
29 29 _EQ_FLOOR = 50.0 # ...and clears this floor, so a wrong-mod fit is never locked in
30 30 _EQ_QAM_FLOOR = 60.0 # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop c
    l, l an
31 31 # drag a lower-order cloud onto a 32/64 lattice at a marginal lock (~51)
    l, l , a
32 32 # confident WRONG order. Genuine multipath recoveries clear ~72; the sweep
    l, l 's
33 33 # eq cases are qpsk/16qam (never 32/64 via eq) so this bar is byte-identic
    l, l al.
34 34 _BAUD_GAIN = 5.0 # an in-band baud hypothesis must beat the global one by this lock
35 35 _SHORT_LOCK = 60.0 # below this, retry the baud line with fewer blocks (short-burst rescue)
36 36 _SYNC_LOCK = 60.0 # below this, try a repeated-preamble carrier/timing sync (packetized bursts
    l, l )
37 37 _ALIAS_MARGIN = 10.0 # a preamble carrier alias must beat the next by this lock, else ambiguous
    l, l .
38 38 # This is the LOAD-BEARING rotation guard: even when the coarse anchor itse
    l, l lf
39 39 # aliases and validates a rotation at ~80 lock, that rotation wins by a thi
    l, l n
40 40 # margin (~8) -- so a 10-lock margin refuses it. Because the margin catche
    l, l s it,
41 41 # the lock floor can stay modest (78, not 82) -> ~5% more genuine recoverie
    l, l s.
42 42 _SYNC_ACCEPT = 78.0 # ...AND clear this absolute lock: below it correct/rotated aliases overlap
    l, l . A
43 43 # 65 floor was tried (to recover more moderate bursts) but an independent
44 44 # snr-12 grid found 8psk rotated aliases locking 71-72 in the opened 65-7
    l, l 8 band
45 45 # (carrier off by one baud/L step -> confident garbage bits). Even 78 leak
    l, l s a
46 46 # a few hard-corner rotations (a separate pre-existing gate limit): hold a
    l, l t 78.
47 47 _SYNC_ADIST = 1.5 # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
48 48 # soft backstop to the margin: it catches the one rotation the margin misse
    l, l s,
49 49 # but tuned LOOSE (1.5, not 0.75) so it does not needlessly reject genuine
50 50 # large-offset recoveries. The three together give 0 confident-wrong across
51 51 # 1920 hard-corner bursts at the max recall this gate reaches.
52 52 _DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"} # pi4dqpsk arrives as qpsk -> dqpsk
53 53
54 54
55 55 @dataclass
56 56 class Result:
57 57     meta: Meta
58 58     family: str # 'linear' | 'fsk'
59 59     burst: tuple[int, int]
60 60     fc: float
61 61     baud: float
62 62     mod: str

```

```

63 63 lock: float
64 64 symbols: np.ndarray          # aligned symbols (linear) / normalized levels (fsk)
65 65 bits: np.ndarray
66 66 iq_corr: np.ndarray          # exportable corrected baseband
67 67 burst_x: np.ndarray = field(repr=False, default=None) # for spectrum views
68 68 symmetry: int = 0
69 69 carrier_ambiguous: bool = False
70 70 baud_conf: float = 0.0
71 71 rolloff: float | None = None
72 72 h: float | None = None      # FSK modulation index
73 73 evm: float = float("nan")
74 74 mer_db: float = float("nan")
75 75 occupied: int = 0
76 76 snr_est: float = float("nan")
77 77 eq_applied: bool = False
78 78 eq_mode: str | None = None  # 'sym' | 'fse'
79 79 alias_resolved: bool = False
80 80 baud_fallback: bool = False
81 81 bursts: list = field(default_factory=list)
82 82 burst_idx: int = 0
83 83 preamble: object = None    # sync.Preamble when a repeated preamble drove the decode
84 84
85 85 def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
86 86     z = np.nan_to_num(self.symbols) # a dead/DC capture -> NaN symbols -> invalid JSON
87 87     if z.size > max_points: # keep TIME ORDER: the UI animates this sequence
88 88         z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
89 89     det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
90 90           "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
91 91           "baud_conf": round(self.baud_conf, 1),
92 92           "alias_resolved": self.alias_resolved,
93 93           "baud_fallback": self.baud_fallback}
94 94     det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
95 95     rf = self.meta.rf_center          # real RF = capture centre + baseband fc
96 96     det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
97 97     if self.h is not None:
98 98         det["h"] = round(self.h, 3)
99 99     if self.preamble is not None:    # a repeated preamble drove the sync
100 100         p = self.preamble
101 101         det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
102 102                           "conf": round(p.conf, 2), "start": p.start, "end": p.end}
103 103     doc = {
104 +     "fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype, # dtype
    +     e 은 한 줄
105 +     "rf_center": self.meta.rf_center,          # 보고에 필수: lock 0 의 1순위 용의자다
106 +
107 107     "family": self.family, "n_samples": int(self.iq_corr.size),
108 108     "burst": {"start": self.burst[0], "end": self.burst[1]},
109 109     "bursts": [{"start": s, "end": e} for s, e in self.bursts],
110 110     "burst_idx": self.burst_idx,
111 111     "detected": det,
112 112     "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
113 113               "evm": _r(self.evm, 4)},
114 114     "snr_est_db": _r(self.snr_est),
115 115     "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
116 116     "constellation": {"i": np.round(z.real, 4).tolist(),
117 117                     "q": np.round(z.imag, 4).tolist()},
118 118     "bits": "".join(map(str, self.bits[:_BITS_CAP])),
119 119     }
120 120     if views and self.burst_x is not None:
121 121         doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)

```

```

120 122         doc["waterfall"] = waterfall(self.burst_x, self.meta.fs)
121 123     return doc
122 124
123 125     def save_iq(self, path: str) -> None:
124 126         np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)
125 127
126 128     def save_symbols(self, path: str) -> None:
127 129         np.save(path, self.symbols.astype(np.complex64))
128 130
129 131     def save_bits(self, path: str, packed: bool = False) -> None:
130 132         if packed:
131 133             np.packbits(self.bits).tofile(path)
132 134         else:
133 135             with open(path, "w") as fh:
134 136                 fh.write("".join(map(str, self.bits)))
135 137
136 138     def save_report(self, path: str) -> None:
137 139         with open(path, "w") as fh:
138 140             json.dump(self.to_json(), fh, indent=1)
139 141
140 142
141 143     def _r(v: float, nd: int = 2) -> float | None:
142 144         return None if v is None or not np.isfinite(v) else round(float(v), nd)
143 145
144 146
145 147     def _fine(z: np.ndarray, mod: str) -> np.ndarray:
146 148         """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
147 149         return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)
148 150
149 151
150 152     def _blocks(n: int) -> int:
151 153         """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
152 154         low SNR) wants fewer, longer segments; only long records need more for
153 155         stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
154 156         return int(np.clip(n // 30000, 1, 8))
155 157
156 158
157 159     def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
158 160         """Matched filter + timing + classify + fine sync at one baud hypothesis."""
159 161         alpha = dsp.est_rolloff(xd, fs, baud)
160 162         ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
161 163         raw = dsp.timing(ym, 4)
162 164         mod = cl.classify(raw, symmetry)
163 165         # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
164 166         # the bulk phase first kills the loop transient that would corrupt the first symbols
165 167         syms = _fine(raw, mod)
166 168         return alpha, ym, raw, mod, syms, cl.quality(syms, mod)
167 169
168 170
169 171     def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
170 172         """Equalize with a seed constellation, then let the OPENED eye pick the mod:
171 173         heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
172 174         with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
173 175         z1 = eq_fn(z, seed_mod)
174 176         cand_s = []
175 177         for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}: # symmetry too, per the docstrin
176 178             g
177 179                 s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
178 180                 cand_s.append((cl.quality(s, m), s, m))
179
180         best = max(cand_s, key=lambda c: c[0].lock)

```



```

179 181 # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
180 182 # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).
181 183 # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
182 184 # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
183 185 # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
184 186 lo = [c for c in cands if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
185 187 if lo and best[2] == "qpsk":
186 188     return lo[0]
187 189 return best
188 190
189 191
190 192 def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
191 193             burst: int | None = None) -> Result:
192 194     """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
193 195     `burst` selects one detected burst (default: the most energetic)."""
194 196     if x.size:
195 197         # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
196 198         # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
197 199         # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
198 200         # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
199 201         # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
200 202         # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
201 203         scale = float(np.percentile(np.abs(x[:16]), 99)) # strided: scale detection needs the
202 204         # BULK magnitude, not every sample -- 16x cheaper on a large direct capture
203 205         if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
204 206             x = x / scale
205 207         x = dsp.analytic(x)
206 208         x = x - x.mean() # DC block: LO leakage otherwise corrupts every estimator
207 209         if x.size < 64: # empty / trivially short: nothing to estimate, and it crashes downstream
208 210             raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
209 211         rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
210 212         if rms and (rms < 1e-6 or rms > 1e6): # only PATHOLOGICAL scale: normalize so the scale-var
211 213             x = x / rms # spots (fsk_gate's CV epsilon, est_carrier's x**p ov
212 214             # underflow) stay invariant. Normal captures rms~0(1
213 215             # untouched -> the acceptance sweep is byte-identical
214 216         bursts = dsp.find_bursts(x, meta.fs)
215 217         if burst is None or not 0 <= burst < len(bursts):
216 218             burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
217 219         s, e = bursts[burst]
218 220         xb = x[s:e] if e - s >= 64 else x
219 221
220 222         # chirp gate -- same conjunctive tie-break as triage.family, so direct analyze / survey_web
221 223         # single mode agree with survey's label: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and
222 224         # would force-demodulate into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass)
223 225         # out of this branch, so only a genuine band-sweep is refused -- never demodulated.
224 226         if is_chirp(xb, meta.fs) and sweeps_band(xb, meta.fs):
225 227             info = analyze_chirp(xb, meta.fs)
226 228             kind = f"LoRa 추정 SF{info['sf']}" if info["sf"] else f"μ={info['mu']:.3g} Hz/s"
227 229             raise ValueError(f"처프(FMCW/CSS) 신호입니다 ({kind}, bw≈{info['bw']:.3g} Hz) - "
228 230                             f"성상도 복조 대상이 아닙니다 (서베이 모드에서 특성 보고)")
229 231
230 232         if fsk_gate(xb, meta.fs):
231 233             r = analyze_fsk(xb, meta.fs)
232 234             sym = np.asarray(r["symbols"], dtype=np.complex128)
233 235             return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
234 236                             sym, r["bits"], xb, burst_x=xb, h=r["h"],

```

```

235 237 bursts=bursts, burst_idx=burst)
236 238
237 239 fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
238 240 fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
239 241 amb = abs(symmetry * fc) > 0.4 * meta.fs
240 242 xd = dsp.mix(xb, meta.fs, fc)
241 243
242 244 baud, conf = dsp.est_baud(xd, meta.fs)
243 245 fell = False
244 246 alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
245 247 bw = dsp.occupied_bw(xd, meta.fs)
246 248 if bw and not (0.72 * bw <= baud <= 1.05 * bw):
247 249     # the global |x|^2 peak disagrees with the occupied band: below alpha-0.1
248 250     # (and under harsh channels) data junk outruns the true line. Demod the
249 251     # in-band hypotheses too and let lock decide, with a clear margin.
250 252     for lo in (0.80, 0.67):
251 253         b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
252 254         if abs(b2 - baud) <= 0.01 * b2:
253 255             continue
254 256         t = _demod(xd, meta.fs, b2, symmetry)
255 257         if t[5].lock > q.lock + _BAUD_GAIN:
256 258             alpha, ym, raw, mod, syms, q = t
257 259             baud, conf, fell = b2, c2, True
258 260
259 261 if q.lock < _SHORT_LOCK:
260 262     # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
261 263     # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
262 264     # carrier _blocks logic, applied to baud). Run both and keep only a clearly-better
263 265     # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
264 266     for nb in (2, 1):
265 267         b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
266 268         if abs(b2 - baud) <= 0.01 * b2:
267 269             continue
268 270         t = _demod(xd, meta.fs, b2, symmetry)
269 271         if t[5].lock > q.lock + _BAUD_GAIN:
270 272             alpha, ym, raw, mod, syms, q = t
271 273             baud, conf, fell = b2, c2, True
272 274
273 275 eq_applied, eq_mode, pre = False, None, None
274 276 if q.lock < _EQ_LOCK: # ISI (multipath) is what a phase-only loop cannot fix
275 277     # CMA is modulus-only, so it opens the eye without knowing the constellation.
276 278     qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
277 279     mode = "sym"
278 280     if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
279 281         # symbol-spaced FIR could not invert the channel; retry T/2-spaced on
280 282         # the clock-tracked 2 sps stream (unaliaised spectrum, longer inverse)
281 283         q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
282 284             dsp.timing(ym, 4, out=2), mod, symmetry)
283 285         if q2.lock > qe.lock:
284 286             qe, se, me, mode = q2, s2, m2, "fse"
285 287     floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
286 288     if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
287 289         syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
288 290 elif mod in ("16qam", "32qam", "64qam"):
289 291     # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal
290 292     # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
291 293     # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
292 294     # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched
293 295     qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)

```

```

294 296 mode = "sym"
295 297 if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FL00R:
296 298     # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
297 299     # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
298 300     # QAM (no lower order) never pays for the FSE.
299 301     qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
300 302         dsp.timing(ym, 4, out=2), mod, symmetry)
301 303     mode = "fse"
302 304 if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FL00R:
303 305     syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
304 306
305 307 if q.lock < _SYNC_LOCK:
306 308     # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
307 309     # carrier/ baud/timing from only a few symbols, where the blind M-th-power estimator (nee
308 310     # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock
309 311     # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
310 312     # reaches here. Detect the preamble, mix by its CF0, demod the DATA past it, keep-best -
311 313     # random / no-preamble signal yields no plateau or a CF0 that does not improve lock an
312 314     # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the b
313 315     # (fs*L/P per block length L), the phase slope gives the carrier (+-fs/P resolves the L-
314 316     # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
315 317     ps = find_preamble(xb, meta.fs, baud_hint=baud)
316 318     if ps is not None:
317 319         # The Schmidl-Cox CF0 is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by bau
318 320         # rotates a whole constellation position per symbol, so the eye still looks locked b
319 321         # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coar
320 322         # M-th-power fc lands within ~1 alias step of the truth even on these short bursts
321 323         # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct a
322 324         # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
323 325         step = meta.fs / ps.period
324 326         k0 = round((fc - ps.cfo_hz) / step)
325 327         cands = []
326 328         for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
327 329             for k in range(k0 - 3, k0 + 4):
328 330                 cfo = ps.cfo_hz + k * step
329 331                 data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
330 332                 if data.size < 256:
331 333                     continue
332 334                 for sm in (2, 4, 8):
333 335                     t = _demod(data, meta.fs, float(b2), sm)
334 336                     cands.append((t[5].lock, cfo, float(b2), sm, t))
335 337                 cands.sort(key=lambda c: -c[0])
336 338                 # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG on
337 339                 # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear g
338 340                 # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rat
339 341                 # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete

```

```

        .)
340 342         # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -
        # - a
341 343         # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock also
        # ne
342 344         # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38)
        # . Two
343 345         # conjunctive conditions favour precision over recall: (1) a strong absolute lock, a
        # nd
344 346         # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -
        # - a
345 347         # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
346 348         # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); o
        # n the
347 349         # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
348 350         if cands and cands[0][0] >= _SYNC_ACCEPT and cands[0][0] > q.lock + _EQ_GAIN:
349 351             top = cands[0]
350 352             other = next((c for c in cands if abs(c[1] - top[1]) > step / 2), None)
351 353             anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
352 354             if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
353 355                 alpha, ym, raw, mod, syms, q = top[4]
354 356                 fc, baud, symmetry = top[1], top[2], top[3]
355 357                 eq_applied, eq_mode, pre = False, None, ps
356 358                 # diagnostics must describe the ACCEPTED decode, not the abandoned blind
357 359                 # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
358 360                 # keeps alias_resolved False), the baud from the preamble period (no
359 361                 # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
360 362                 fc0, conf, fell = fc, 0.0, False
361 363                 amb = abs(symmetry * fc) > 0.4 * meta.fs
362 364
363 365         bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
364 366             else demap_bits(syms, mod)
365 367         return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
366 368             burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
367 369             rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
368 370             snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
369 371             alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
370 372             bursts=bursts, burst_idx=burst, preamble=pre)
371 373
372 374
373 375 def analyze_file(path: str, fs: float | None = None, fmt: str | None = None,
374 376     dtype: str | None = None, endian: str | None = None,
375 377     bitrev: bool | None = None, diff: bool = False,
376 378     burst: int | None = None, rf: float | None = None) -> Result:
377 379     """Analyze a file; explicit args override SigMF/filename tokens."""
378 380     from .sigio import parse_sigmf
379 381     m = parse_sigmf(path) or parse_name(path)
380 382     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
381 383         endian or m.endian, m.bitrev if bitrev is None else bitrev,
382 384         rf_center=rf if rf is not None else m.rf_center)
383 385     x, meta = read(path, meta)
384 386     return analyze(x, meta, diff, burst)
385 387
386 388
387 389 # --- wideband survey: many emitters in one capture -----
388 390
389 391 @dataclass
390 392 class Emitter:
391 393     detection: Detection
392 394     kind: str                # linear|fsk|chirp|analog|tone|tooshort|error

```

```

393 395     abs_fc: float                # emitter carrier vs capture centre (Hz)
394 396     result: Result | None = None # demod result for digital kinds
395 397     info: dict | None = None     # characterization for non-digital kinds (e.g. chirp params)
396 398
397 399     def to_json(self) -> dict:
398 400         d = self.detection
399 401         doc = {"kind": self.kind, "abs_fc": round(self.abs_fc, 3),
400 402                "det": {"fc": round(d.fc, 3), "bw": round(d.bw, 3),
401 403                        "t0": d.t0, "t1": d.t1, "snr_db": _r(d.snr_db),
402 404                        "baud_hint": round(d.baud_hint, 1)}}
403 405         if self.info is not None:
404 406             doc["info"] = self.info
405 407         if self.result is not None:
406 408             r = self.result
407 409             doc.update(mod=r.mod, baud=round(r.baud, 3), lock=round(r.lock, 1),
408 410                        mer_db=_r(r.mer_db), evm=_r(r.evm, 4), family=r.family)
409 411         return doc
410 412
411 413     def to_detail(self, rf_center: float | None = None) -> dict:
412 414         """Web drill-down payload: the box summary plus, for a digital emitter, the
413 415         full analyze result so the UI reuses its single-signal detail view verbatim.
414 416         The channel is demodulated at baseband (no rf_center), and r.fc is only the
415 417         residual within-channel offset -- so the emitter's absolute RF is the capture
416 418         centre plus abs_fc (the full offset from capture centre), injected here."""
417 419         doc = self.to_json()
418 420         if self.result is not None:
419 421             doc["result"] = self.result.to_json()
420 422             if rf_center is not None:
421 423                 doc["result"]["detected"]["rf_hz"] = round(rf_center + self.abs_fc, 3)
422 424         return doc
423 425
424 426
425 427 @dataclass
426 428 class Survey:
427 429     meta: Meta
428 430     emitters: list[Emitter] = field(default_factory=list)
429 431
430 432     def to_json(self) -> dict:
431 433         return {"fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype,
432 434                "n_emitters": len(self.emitters),
433 435                "emitters": [e.to_json() for e in self.emitters]}
434 436
435 437
436 438 def survey(x: np.ndarray, meta: Meta, *, diff: bool = False, **detect_kw) -> Survey:
437 439     """Detect every emitter in a wideband capture, extract each to its own baseband
438 440     channel, and demodulate the digital ones with the unchanged single-signal chain.
439 441     A capture holding one signal that fills the band collapses to a single emitter."""
440 442     xa = dsp.analytic(x)
441 443     xa = xa - xa.mean()
442 444     emitters = []
443 445     for d in detect(xa, meta.fs, **detect_kw):
444 446         ch, fs_ch = extract(xa, meta.fs, d)
445 447         if ch.size < 256:
446 448             emitters.append(Emitter(d, "tooshort", d.fc))
447 449             continue
448 450         kind = triage.family(ch, fs_ch)
449 451         if kind in ("linear", "fsk"):
450 452             # triage only gates digital vs not; analyze's own family is authoritative.
451 453             # Isolate each channel: a degenerate one must not abort the whole survey.
452 454             try:

```

```

453 455         r = analyze(ch, Meta(fs_ch, "iq", "f32", "le", False), diff=diff)
454 456         emitters.append(Emitter(d, r.family, d.fc + r.fc, r))
455 457     except Exception:
456 458         emitters.append(Emitter(d, "error", d.fc))
457 459     else:
458 460         info = analyze_chirp(ch, fs_ch) if kind == "chirp" else None
459 461         emitters.append(Emitter(d, kind, d.fc, info=info))
460 462     return Survey(meta, emitters)
461 463
462 464
463 465 def survey_web(x: np.ndarray, meta: Meta, *, diff: bool = False) -> dict:
464 466     """Web survey payload for /api/survey. When detect sees <=1 emitter the capture
465 467     is effectively one signal -> 'single' mode returns the UNCHANGED direct analyze()
466 468     result (correct fc, matches /api/analyze). Otherwise 'survey' mode adds a whole-
467 469     capture overview waterfall and every emitter's drill-down detail. Additive: does
468 470     not alter survey()/Survey/Emitter, so CLI + existing tests are unaffected."""
469 471     xa = dsp.analytic(x)
470 472     xa = xa - xa.mean()
471 473     if len(detect(xa, meta.fs)) <= 1:
472 474         r = analyze(x, meta, diff=diff)
473 475         out = {"mode": "single", "result": r.to_json()}
474 476         if len(r.bursts) > 1: # multi-burst signal -> whole-record map for burst selection
475 477             out["overview"] = {"n": int(xa.size), "fs": meta.fs,
476 478                                "waterfall": waterfall(xa, meta.fs)}
477 479         return out
478 480     sv = survey(x, meta, diff=diff)
479 481     return {"mode": "survey", "fs": meta.fs, "fmt": meta.fmt, "rf_center": meta.rf_center,
480 482            "overview": {"fs": meta.fs, "n": int(xa.size), "spectrum": spectrum(xa, meta.fs),
481 483                        "waterfall": waterfall(xa, meta.fs)},
482 484            "emitters": [e.to_detail(meta.rf_center) for e in sv.emitters]}
483 485
484 486
485 487 def survey_file(path: str, fs: float | None = None, fmt: str | None = None,
486 488                 dtype: str | None = None, endian: str | None = None,
487 489                 bitrev: bool | None = None, diff: bool = False,
488 490                 rf: float | None = None) -> Survey:
489 491     """Survey a capture file; explicit args override SigMF/filename tokens."""
490 492     from .sigio import parse_sigmf
491 493     m = parse_sigmf(path) or parse_name(path)
492 494     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
493 495                endian or m.endian, m.bitrev if bitrev is None else bitrev,
494 496                rf_center=rf if rf is not None else m.rf_center)
495 497     x, meta = read(path, meta)
496 498     return survey(x, meta, diff=diff)

```

signus/cli.py +74 -276 · 코드북 44쪽 · 새로 쓸 줄 1-2, 5-6, 8, 10-11, 15-26, 29-35, 38-64, 67, 73-74, 104-105, 141-142, 154-155, 183-189, 196-200

```

1 + """CLI: analyze FILE / survey / serve. 합성·채점(gen/dataset/sweep)은 개발기 전용 lab.py 가
2 + 단다 — 격리망 장비에는 그 파일이 없어 명령 자체가 없다."""

```

```

2 3
3 4 import argparse
5 + import importlib.util
6 + import re
4 7 import sys
8 + from zlib import crc32
5 9
10 + from .pipeline import analyze_file, survey_file
11 + from .sigio import sidecar_read
22 12
23 13 _DTYPE_CHOICES = ("i8", "u8", "i16", "u16", "f32", "f64")
24 14

```

```

15 +
16 + # --- 손으로 옮기는 한 줄 -----
17 + # 실신호 장비는 인터넷도 클립보드도 없다. 결과는 사람이 화면을 보고 받아쳐서 나온다. 그래서
18 + # 이 블록은 반드시 여기(필사 대상)에 있어야 한다 -- tools/ 에 두면 그 도구부터 필사해야 하는
19 + # 분말전도가 된다. 줄 끝의 검출 코드가 "옮기다 한 글자 틀림"을 잡는다.
20 +
21 + _ALPHA = "0123456789abcdefghijklmnopqrstuvwxyz" # 0/o, 1/l/i 처럼 화면에서 헷갈리는 글자를 뺀 32자
22 + _BRIEF_FLAGS = [("eq", lambda d: d["eq"]["applied"]),
23 +                ("al", lambda d: d["detected"]["alias_resolved"]),
24 +                ("fb", lambda d: d["detected"]["baud_fallback"]),
25 +                ("amb", lambda d: d["detected"]["carrier_ambiguous"]),
26 +                ("pre", lambda d: "preamble" in d["detected"])]
27 27
28 28
29 + def check_code(text: str) -> str:
30 +     """받아친 줄의 오타 검출 코드 (4글자 = 20비트). 공백은 '한 칸으로 줄이되 없애지는'
31 +     않는다 -- 전부 지우면 'fs1000000 16qam' 과 'fs10000001 6qam' (샘플레이트 10배!) 이 같은
32 +     코드가 되어, 값이 바뀌는 오타 32종이 조용히 통과했다. 대소문자와 줄 간격은 계속 무시한다."""
33 +     body = re.sub(r"\s+", " ", re.sub(r"#[0-9a-z]{4}\b", "", text.lower())).strip()
34 +     n = crc32(body.encode())
35 +     return "".join(_ALPHA[(n >> (5 * i)) & 31] for i in (3, 2, 1, 0))
55 36
56 37
38 + def brief(doc: dict, mode: str) -> str:
39 +     """손으로 옮기는 한 줄 + 검출 코드. Result.to_json() 디렉터리에서 만든다 -- 객체
40 +     속성을 직접 포맷하면 to_json 이 이미 한 반올림과 두 번 겹쳐 장비와 맥이 갈린다."""
41 +     head = f"sig2 {mode} fs{doc['fs']:.0f} {doc['fmt']}-{doc['dtype']}"
42 +     if mode == "sv":
43 +         lines = [f"{head} n{doc['n_emitters']}"]
44 +         for i, e in enumerate(doc["emitters"][:12]):
45 +             baud = f" bd{e['baud']:.0f}" if e.get("baud") else ""
46 +             lock = f" lk{e['lock']:.0f}" if e.get("lock") is not None else ""
47 +             lines.append(f"{i} fc{e['abs_fc']:.0f}{baud}{lock} {e.get('mod') or e['kind']}")
48 +         if len(doc["emitters"]) > 12:
49 +             lines.append(f"...{len(doc['emitters']) - 12}개 생략")
50 +     else:
51 +         d, q = doc["detected"], doc["quality"]
52 +         p = [head, d["mod"], f"fc{d['fc']:.0f}", f"bd{d['baud']:.0f}", f"lk{q['lock']:.0f}"]
53 +         if q["mer_db"] is not None:
54 +             p.append(f"mer{q['mer_db']:.1f}")
55 +         if d.get("h") is not None: # FSK 는 롤오프 대신 변조지수
56 +             p.append(f"h{d['h']:.2f}")
57 +         elif d["rolloff"] is not None:
58 +             p.append(f"rl{d['rolloff']:.2f}")
59 +         if len(doc["bursts"]) > 1:
60 +             p.append(f"b{doc['burst_idx'] + 1}/{len(doc['bursts'])}")
61 +         p += [f for f, get in _BRIEF_FLAGS if get(doc)]
62 +         lines = [" ".join(p)]
63 +         body = "\n".join(lines)
64 +         return f"{body} #{check_code(body)}"
64 65
65 66
67 + # --- analyze / survey / serve -----
201 68
202 69 def _analyze(args: argparse.Namespace) -> int:
203 70     r = analyze_file(args.file, args.fs, args.fmt, args.dtype,
204 71                     "be" if args.be else None, args.bitrev or None, args.diff,
205 72                     None if args.burst is None else args.burst - 1, rf=args.rf)
73 +     j = r.to_json(views=False) # 한 줄 요약도 여기서 만든다 (반올림이 한 번만 걸리게)
74 +     d = j["detected"]

```

```

207 75     truth = (sidecar_read(args.file) or {}).get("truth") # display only, never detection
208 76     if len(r.bursts) > 1:
209 77         print(f"버스트 {len(r.bursts)}개 감지 - {r.burst_idx + 1}번째 분석 (--burst N 으로 선택)
        ")
210 78     print(f"모드      {d['mod']})" + (f"      (정답 {truth['mod']})" if truth else "")
211 79     print(f"반송파      {d['fc']:.1f} Hz" + (f"      (정답 {truth['fc']:.1f})" if truth else ""))
212 80     if d["rf_hz"] is not None:
213 81         print(f"실제 RF      {d['rf_hz'] / 1e6:.6f} MHz")
214 82     print(f"심볼레이트 {d['baud']:.1f} Hz" + (f"      (정답 {truth['baud']:.1f})" if truth else ""))
        )
215 83     fsk = r.family == "fsk"
216 84     tail = f"변조지수 h {d['h']:.2f}" if fsk else f"롤오프      {d['rolloff']:.2f}"
217 85     mer = "" if fsk else f"      MER {r.mer_db:.1f} dB"
218 86     print(f"{tail}      lock {r.lock:.1f}{mer}")
219 87     if r.eq_applied:
220 88         print("등화기 적용: 다중경로 ISI 보정됨"
221 89             + (" (T/2 분수간격)" if r.eq_mode == "fse" else " (심볼간격)"))
222 90     if r.alias_resolved:
223 91         print("반송파 앨리어스 보정: 스펙트럼 중심으로 후보 선택")
224 92     if r.baud_fallback:
225 93         print("심볼레이트 폴백: 점유대역폭 사전정보로 재탐색")
226 94     if r.carrier_ambiguous:
227 95         print("경고: 반송파 앨리어싱 가능 (|sym*fc| > 0.4*fs)")
228 96     if args.save_iq:
229 97         r.save_iq(args.save_iq)
230 98     if args.save_symbols:
231 99         r.save_symbols(args.save_symbols)
232 100    if args.save_bits:
233 101        r.save_bits(args.save_bits, args.packed)
234 102    if args.report:
235 103        r.save_report(args.report)
104 +    if args.brief:                                # 사람용 출력을 대체하지 않고 맨 끝에 한 줄 더 --
105 +        print(brief(j, "an")) # 조기 return 이면 위 저장들이 조용히 무시된다
236 106    return 0
237 107
238 108
239 109 def _fhz(v: float) -> str:
240 110     a = abs(v)
241 111     if a >= 1e6:
242 112         return f"{v / 1e6:+.3f} MHz"
243 113     if a >= 1e3:
244 114         return f"{v / 1e3:+.2f} kHz"
245 115     return f"{v:+.0f} Hz"
246 116
247 117
248 118 def _survey(args: argparse.Namespace) -> int:
249 119     """Wideband survey: detect every emitter, demodulate the digital ones."""
250 120     s = survey_file(args.file, args.fs, args.fmt, args.dtype,
251 121         "be" if args.be else None, args.bitrev or None, args.diff, rf=args.rf)
252 122     rf0 = s.meta.rf_center
253 123     note = f" · RF 중심 {rf0 / 1e6:.3f} MHz" if rf0 is not None else ""
254 124     print(f"{len(s.emitters)}개 신호 감지 (샘플레이트 {s.meta.fs:.0f} Hz{note})")
255 125     print(f"{'#':>2} {'실제 RF' if rf0 is not None else '중심주파수':>12} {'대역폭':>10}"
256 126         f" {'분류':>7} {'변조':>7} {'심볼레이트':>11} {'lock':>5}")
257 127     for i, e in enumerate(s.emitters):
258 128         r = e.result
259 129         mod = r.mod if r else "-"
260 130         baud = f"{r.baud:.0f}" if r else "-"
261 131         lock = f"{r.lock:.0f}" if r else "-"
262 132         kind = {"linear": "디지털", "fsk": "FSK", "analog": "아날로그",

```



```

263 133         "tone": "순수톤", "tooshort": "너무짧음", "error": "오류"}.get(e.kind, e.kind)
264 134         fc = e.abs_fc if rf0 is None else rf0 + e.abs_fc
265 135         print(f"{i:>2} {_fhz(fc):>12} {_fhz(e.detection.bw):>10} {kind:>7}")
266 136         f" {mod:>7} {baud:>11} {lock:>5}")
267 137     if args.report:
268 138         import json
269 139         with open(args.report, "w") as fh:
270 140             json.dump(s.to_json(), fh, indent=1)
141 +     if args.brief:
142 +         print(brief(s.to_json(), "sv"))
320 143     return 0
321 144
322 145
323 146 def _read_args(p: argparse.ArgumentParser) -> None:
324 147     """Read-side sample-format overrides, shared by analyze/survey (sidecar/filename win)."""
325 148     p.add_argument("--fs", type=float)
326 149     p.add_argument("--fmt", choices=("iq", "real"))
327 150     p.add_argument("--dtype", choices=DTYPE_CHOICES)
328 151     p.add_argument("--be", action="store_true", help="big-endian 샘플")
329 152     p.add_argument("--bitrev", action="store_true", help="바이트 내 비트 역순")
330 153     p.add_argument("--rf", type=float, help="RF 중심주파수 (Hz) - 실제 주파수로 보고")
154 +     p.add_argument("--brief", action="store_true",
155 +                     help="손으로 옮길 한 줄 + 오타 검출 코드를 끝에 덧붙임")
331 156
332 157
333 158 def main(argv: list[str] | None = None) -> int:
334 159     ap = argparse.ArgumentParser(prog="signus")
335 160     sub = ap.add_subparsers(dest="cmd", required=True)
336 161
337 162     a = sub.add_parser("analyze", help="블라인드 복조 + 제원 탐지")
338 163     a.add_argument("file")
339 164     _read_args(a)
340 165     a.add_argument("--save-iq")
341 166     a.add_argument("--save-symbols")
342 167     a.add_argument("--save-bits")
343 168     a.add_argument("--packed", action="store_true")
344 169     a.add_argument("--report")
345 170     a.add_argument("--diff", action="store_true",
346 171                     help="차동 디맵(D-BPSK/D-QPSK): 회전 모호성 없이 비트 복원")
347 172     a.add_argument("--burst", type=int, help="분석할 버스트 번호 (1부터; 기본 최강 버스트)")
348 173
349 174     sv = sub.add_parser("survey", help="광대역 캡처의 모든 신호 탐지 + 복조")
350 175     sv.add_argument("file")
351 176     _read_args(sv)
352 177     sv.add_argument("--diff", action="store_true", help="차동 디맵")
353 178     sv.add_argument("--report", help="JSON 리포트 저장 경로")
354 179
385 180     s = sub.add_parser("serve", help="웹 UI 서버")
386 181     s.add_argument("--host", default="127.0.0.1")
387 182     s.add_argument("--port", type=int, default=8000)
183 +
184 +     # 합성 생성·채점(gen/dataset/sweep)은 개발기 전용이다. 격리망 장비에는 lab.py 를
185 +     # 옮기지 않으므로 그 장비에서는 이 명령들이 아예 존재하지 않는다. try/except 로 삼키면
186 +     # 개발기에서 lab 내부의 진짜 임포트 오류까지 "명령 없음"으로 둔갑하니 파일 유무만 본다.
187 +     if importlib.util.find_spec("signus.lab"):
188 +         from . import lab
189 +         lab.add_commands(sub)
388 190
389 191     args = ap.parse_args(argv)
390 192     if args.cmd == "analyze":

```

```

391 193     return _analyze(args)
392 194     if args.cmd == "survey":
393 195         return _survey(args)
196 +     if args.cmd == "serve":
197 +         from .server import run
198 +         run(args.host, args.port)
199 +         return 0
200 +     return args.run(args)          # lab 이 단 명령 (gen / dataset / sweep)

```

```

403 201
404 202
405 203     if __name__ == "__main__":
406 204         sys.exit(main())

```

signus/web/app.js +31 -11 · 코드북 59쪽 · 새로 쓸 줄 5-6, 15-18, 20-31, 33-34, 36-37, 107-115

```

1 1     "use strict";
2 2     const $ = (id) => document.getElementById(id);
3 3     const API = "/api/analyze";
4 4     const SURVEY_API = "/api/survey";
5 +     const FS_RE = /(?:^|[\._])fs(\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[._]|$)/i;
6 +     const RF_RE = /(?:^|[\._])rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[._]|$)/i;
7 7     const REDUCED = matchMedia("(prefers-reduced-motion: reduce)").matches;
8 8     const state = { file: null, sidecar: null, meta: null, resp: null, batch: [],
9 9                     burst: null, survey: null, drilled: false, overview: null };
10 10
11 11     /* --- filename parsing (mirror of sigio.parse_name; read-necessities only) --- */
12 12     const DTYPES = ["i8", "u8", "i16", "u16", "f32", "f64"];
13 13     // baudline-style aliases: <bits>t = two's complement (signed), <bits>o = offset (unsigned)
14 14     const DTYPE_ALIAS = { "8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "f64" };
15 15     const FMTS = { cplx: "iq", real: "real", iq: "iq" }; // iq = 옛 밀줄 형식의 이름
16 +     /* <이름>.cplx|real.<샘플레이트>.<16t>.pcm - 샘플레이트는 접두사가 없으므로 포맷 바로
17 +     다음 자리로 찾는다. 옛 밀줄 형식(<이름>_fs..._iq_i16.iq)도 그대로 읽는다. sigio.py 와 규칙이
18 +     같아야 한다: 여기서 갈리면 웹은 되고 CLI 는 안 되는(또는 그 반대) 조용한 불일치가 된다. */
19 19     function parseName(name) {
20 +     const base = name.replace(/^.*\\/, "").toLowerCase();
21 +     const mt = FS_RE.exec(base), rt = RF_RE.exec(base), toks = base.split(/[._]/);
22 +     // 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 마지막 후보. 라벨의 real/cplx/
23 +     // u8/be 오염과 점 낀 샘플레이트(2.4e6 -> 2 Hz)를 같은 관문으로 막는다 (sigio.py 와 동일 규칙)
24 +     const spec = (t) => DTYPES.includes(t) || t in DTYPE_ALIAS || t === "be" || t === "bitrev"
25 +     || t === "pcm" || t.startsWith("rf");
26 +     const hit = (k) => /^\\d+$/ .test(toks[k + 1] || "") && spec(toks[k + 2] || "");
27 +     const cand = toks.map((t, k) => (t in FMTS ? k : -1)).filter((k) => k >= 0);
28 +     const hits = cand.filter(hit);
29 +     const i = hits.length ? hits[hits.length - 1] : (cand.length ? cand[0] : -1);
30 +     const tail = i >= 0 ? toks.slice(i + 1) : toks; // 제원은 포맷 토막 뒤에만 산다
31 +     const dt = tail.find((t) => DTYPES.includes(t) || t in DTYPE_ALIAS);
32 32     return {
33 +     fs: hits.includes(i) ? parseFloat(toks[i + 1]) : (mt ? parseFloat(mt[1]) : null),
34 +     fmt: i >= 0 ? FMTS[toks[i]] : null,
35 35     dtype: dt ? (DTYPE_ALIAS[dt] || dt) : "i16",
36 +     endian: tail.includes("be") ? "be" : "le",
37 +     bitrev: tail.includes("bitrev"),
38 38     rf: rt ? parseFloat(rt[1]) : null,
39 39     };
40 40 }
41 41     /* SigMF: core:datatype -> our fmt/dtype/endian */
42 42     const SIGMF = { cf32: ["iq", "f32"], ci16: ["iq", "i16"], ci8: ["iq", "i8"],
43 43                     cu8: ["iq", "u8"], rf32: ["real", "f32"], ri16: ["real", "i16"] };
44 44     function metaFromSigmf(doc) {

```

```

32 45   const g = doc && doc.global;
33 46   if (!g || !g["core:datatype"]) return null;
34 47   const dt = g["core:datatype"], base = dt.replace(/_[lb]e$/, "");
35 48   if (!(base in SIGMF)) return null;
36 49   const [fmt, dtype] = SIGMF[base];
37 50   const cap = (doc.captures || [])[0] || {};
38 51   return { fs: Number(g["core:sample_rate"]), fmt, dtype,
39 52           endian: dt.endsWith("_be") ? "be" : "le", bitrev: false,
40 53           rf: cap["core:frequency"] != null ? Number(cap["core:frequency"]) : null };
41 54 }
42 55
43 56 /* --- number formatting --- */
44 57 function sig(x) {
45 58   const a = Math.abs(x);
46 59   return parseFloat(a >= 100 ? x.toFixed(0) : a >= 10 ? x.toFixed(1) : x.toFixed(2)).toString();
47 60 }
48 61 function fmtHz(v) {
49 62   if (v == null) return "-";
50 63   const a = Math.abs(v);
51 64   if (a >= 1e6) return sig(v / 1e6) + " MHz";
52 65   if (a >= 1e3) return sig(v / 1e3) + " kHz";
53 66   return sig(v) + " Hz";
54 67 }
55 68 function fmtRf(v) { // absolute RF: keep kHz resolution even in the 100s-of-MHz range
56 69   return (v / 1e6).toFixed(3) + " MHz";
57 70 }
58 71
59 72 /* --- ideal constellation points (mirror of constellations.ideal_points) --- */
60 73 function idealPoints(mod) {
61 74   if (mod === "bpsk") return [{ i: 1, q: 0 }, { i: -1, q: 0 }];
62 75   if (mod === "qpsk" || mod === "8psk") {
63 76     const m = mod === "qpsk" ? 4 : 8, off = mod === "qpsk" ? Math.PI / 4 : 0, p = [];
64 77     for (let k = 0; k < m; k++) p.push({ i: Math.cos(off + 2 * Math.PI * k / m), q: Math.sin(of
65 78       f + 2 * Math.PI * k / m) });
66 79     return p;
67 80   }
68 81   const n = mod === "16qam" ? 4 : mod === "32qam" ? 6 : 8, lv = [], p = [];
69 82   for (let k = 0; k < n; k++) lv.push(k * 2 - (n - 1));
70 83   for (const qi of lv) for (const ii of lv) {
71 84     if (mod === "32qam" && Math.abs(ii * qi) === 25) continue; // cross: corners removed
72 85     p.push({ i: ii, q: qi });
73 86   }
74 87   let e = 0;
75 88   for (const pt of p) e += pt.i * pt.i + pt.q * pt.q;
76 89   const norm = Math.sqrt(e / p.length);
77 90   return p.map((pt) => ({ i: pt.i / norm, q: pt.q / norm }));
78 91 }
79 92 /* --- file intake: one data file (+sidecar) or many (batch) --- */
80 93 async function acceptFiles(list) {
81 94   const files = [...list];
82 95   const metas = files.filter((f) => /\. (json|sigmf-meta)$/i.test(f.name));
83 96   const data = files.filter((f) => !metas.includes(f));
84 97   if (!data.length) return showError("데이터 파일을 찾을 수 없어요.");
85 98   if (data.length > 1) return runBatch(data, metas);
86 99
87 100   state.file = data[0];
88 101   state.resp = null;
89 102   state.batch = [];
90 103   state.burst = null;

```

```

91 104 | state.survey = null;
92 105 | state.drilled = false;
93 106 | state.overview = null;
107 + | // 사이드카는 이름이 같은 캡처의 것만 쓴다 -- 확장자만 보고 집으면 다른 신호의 fs/fmt 로
108 + | // 조용히 읽는다 (일괄 모드는 이미 stem 대조를 한다: 두 모드가 다르면 그게 또 함정이다)
109 + | const mine = (m, ext) => {
110 + |   const stem = m.name.toLowerCase().slice(0, -ext.length);
111 + |   const dn = state.file.name.toLowerCase();
112 + |   return dn.startsWith(stem) && (dn.length === stem.length || dn[stem.length] === ".");
113 + | };
114 + | const sm = metas.find((m) => m.name.toLowerCase().endsWith(".sigmf-meta") && mine(m, ".sigmf-m
  + | eta));
115 + | const truth = metas.find((m) => m.name.toLowerCase().endsWith(".json") && mine(m, ".json"));
96 116 | state.sidecar = truth ? await readJson(truth) : null; // client-side only, never sent
97 117 | state.meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(state.file.name);
98 118 | if (state.meta && state.meta.fs && state.meta.fmt) runSurvey();
99 119 | else showMetaForm();
100 120 | }
101 121 | const readJson = (f) => f.text().then(JSON.parse).catch(() => null);
102 122 |
103 123 | function showMetaForm() {
104 124 |   hideAll();
105 125 |   state.meta = state.meta || {};
106 126 |   if (state.meta.fs) $("mFs").value = state.meta.fs;
107 127 |   $("mFmt").value = state.meta.fmt || "";
108 128 |   $("mDtype").value = state.meta.dtype || "i16";
109 129 |   show($("metaForm"));
110 130 | }
111 131 | $("metaGo").onclick = () => {
112 132 |   const fs = parseFloat($("mFs").value), fmt = $("mFmt").value;
113 133 |   if (!(fs > 0) || !fmt) return showError("샘플레이트와 포맷을 입력해주세요.");
114 134 |   state.meta = { fs, fmt, dtype: $("mDtype").value, endian: "le", bitrev: false };
115 135 |   runSurvey();
116 136 | };
117 137 |
118 138 | /* --- server call (contract-frozen) --- */
119 139 | function query(file, m, burst) {
120 140 |   const q = new URLSearchParams({ name: file.name });
121 141 |   if (m.fs) q.set("fs", m.fs);
122 142 |   if (m.fmt) q.set("fmt", m.fmt);
123 143 |   if (m.dtype) q.set("dtype", m.dtype);
124 144 |   if (m.endian) q.set("endian", m.endian);
125 145 |   if (m.bitrev) q.set("bitrev", "1");
126 146 |   if (m.rf != null) q.set("rf", m.rf);
127 147 |   if (burst != null) q.set("burst", burst);
128 148 |   return q;
129 149 | }
130 150 | async function postTo(api, file, m, burst) {
131 151 |   const res = await fetch(api + "?" + query(file, m, burst), { method: "POST", body: file });
132 152 |   const data = await res.json();
133 153 |   if (!res.ok || data.error) throw new Error(data.error || `서버 오류 (${res.status})`);
134 154 |   return data;
135 155 | }
136 156 | async function runSurvey() { // entry: one capture -> single detail OR survey
137 157 |   hideAll();
138 158 |   state.drilled = false;
139 159 |   $("loadMsg").textContent = "신호를 조사하고 있어요.";
140 160 |   show($("loading"));
141 161 |   try {
142 162 |     const resp = await postTo(SURVEY_API, state.file, state.meta);

```

```

143 163     if (resp.mode === "single") {
144 164         state.survey = null; state.overview = resp.overview || null; state.resp = resp.result;
145 165         render(resp.result);
146 166         if (state.overview) renderBurstMap(state.overview, resp.result);
147 167     } else renderSurvey(resp);
148 168 } catch (e) {
149 169     showError(e.message);
150 170 }
151 171 }
152 172 async function analyze() {                                // burst re-selection within a single signal
153 173     hideAll();
154 174     $("loadMsg").textContent = "신호를 분석하고 있어요...";
155 175     show($("loading"));
156 176     try {
157 177         state.resp = await postTo(API, state.file, state.meta, state.burst);
158 178         render(state.resp);
159 179         if (state.overview) renderBurstMap(state.overview, state.resp);
160 180         $("backBatch").classList.toggle("hidden", !state.batch.length);
161 181     } catch (e) {
162 182         showError(e.message);
163 183     }
164 184 }
165 185
166 186 /* --- batch mode --- */
167 187 async function runBatch(data, metas) {
168 188     hideAll();
169 189     show($("loading"));
170 190     const truths = new Map(metas.filter((m) => /\.json$/i.test(m.name))
171 191         .map((m) => [m.name.replace(/\.json$/i, ""), m]));
172 192     const sigmfs = new Map(metas.filter((m) => /\.sigmf-meta$/i.test(m.name))
173 193         .map((m) => [m.name.replace(/\.sigmf-meta$/i, ""), m]));
174 194     const rows = [];
175 195     for (let i = 0; i < data.length; i++) {
176 196         const f = data[i];
177 197         $("loadMsg").textContent = `일괄 분석 중... (${i + 1}/${data.length}) ${f.name}`;
178 198         // same meta precedence as single-file mode: a .sigmf-meta sidecar (matched by stem)
179 199         // beats filename tokens
180 200         const sm = sigmfs.get(f.name.replace(/\.([^.]+)$/, ""));
181 201         const meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(f.name);
182 202         let resp = null, err = null;
183 203         try {
184 204             if (!meta.fs || !meta.fmt) throw new Error("파일명에 fs/포맷 정보가 없어요");
185 205             resp = await postTo(API, f, meta);
186 206         } catch (e) { err = e.message; }
187 207         const tf = truths.get(f.name);
188 208         // meta is stored per row: a later burst-chip re-analyze posts with THIS file's meta --
189 209         // it used to read state.meta, which a fresh batch never set (crash) and a previous
190 210         // single-file run left stale (silent wrong-fs decode)
191 211         rows.push({ file: f, meta, resp, err, sidecar: tf ? await readJson(tf) : null });
192 212     }
193 213     state.batch = rows;
194 214     hideAll();
195 215     renderBatch(rows);
196 216 }
197 217
198 218 function renderBatch(rows) {
199 219     $("batchBody").innerHTML = rows.map((r, i) => {
200 220         if (r.err) return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
201 221             `<td colspan="4" style="color:var(--red)">${r.err}</td></tr>`;
202 222         const d = r.resp.detected, lock = Math.round(r.resp.quality.lock);

```

```

203 223     const cls = lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad";
204 224     return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
205 225         `<td>${d.mod.toUpperCase()}</td><td>${fmtHz(d.fc)}</td><td>${fmtHz(d.baud)}</td>` +
206 226         `<td><span class="pill ${cls}">${lock}</span></td></tr>`;
207 227   }).join("");
208 228   $("batchBody").querySelectorAll("tr").forEach((tr) => {
209 229     tr.onclick = () => {
210 230       const r = state.batch[+tr.dataset.i];
211 231       if (!r.resp) return;
212 232       state.file = r.file; state.meta = r.meta; state.resp = r.resp; state.sidecar = r.sidecar;
213 233       state.burst = null; state.drilled = false; state.survey = null; state.overview = null;
214 234       hideAll();
215 235       render(r.resp);
216 236       const b = $("backBatch");
217 237       b.textContent = "← 목록으로";
218 238       b.onclick = () => { hideAll(); renderBatch(state.batch); };
219 239       b.classList.remove("hidden");
220 240     };
221 241   });
222 242   show($("batchCard"));
223 243 }
224 244 $("backBatch").onclick = () => { hideAll(); renderBatch(state.batch); };
225 245 $("dlCsv").onclick = () => {
226 246   const head = "file,mod,fc_hz,baud_hz,rolloff,lock,mer_db,snr_est_db\n";
227 247   const body = state.batch.filter((r) => r.resp).map((r) => {
228 248     const d = r.resp.detected, q = r.resp.quality;
229 249     return [r.file.name, d.mod, d.fc, d.baud, d.rolloff ?? "", q.lock, q.mer_db,
230 250         r.resp.snr_est_db ?? ""].join(",");
231 251   }).join("\n");
232 252   saveBlob("signus_batch.csv", head + body, "text/csv");
233 253 };
234 254
235 255 /* --- wideband survey overview (waterfall + detected-emitter boxes) --- */
236 256 function renderSurvey(resp) {
237 257   hideAll();
238 258   stopPlay();
239 259   state.survey = resp;
240 260   const dig = resp.emitters.filter((e) => e.result).length;
241 261   $("survCount").textContent = `${resp.emitters.length}개 신호 · 디지털 ${dig}`;
242 262   $("survInfo").classList.add("hidden");
243 263   renderSurveyList(resp);
244 264   show($("surveyCard")); // show first so the canvas has a real width
245 265   paintWaterfall($("survFall"), resp.overview.waterfall, 230);
246 266   drawBoxes(resp);
247 267 }
248 268
249 269 function boxStyle(e) { // -> [css class, short label]
250 270   if (e.result) {
251 271     const lock = Math.round(e.result.quality.lock);
252 272     return [lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad",
253 273         `${e.result.detected.mod.toUpperCase()} · ${lock}`];
254 274   }
255 275   if (e.kind === "chirp") {
256 276     return ["gray", e.info && e.info.sf ? `LoRa SF${e.info.sf}` : "처프"];
257 277   }
258 278   return ["gray", { analog: "아날로그", tone: "톤/CW", tooshort: "짧음",
259 279       error: "오류" }[e.kind] || e.kind];
260 280 }
261 281
262 282 function drawBoxes(resp) {

```

```

263 283   const host = $("survBoxes"), fs = resp.overview.fs, n = resp.overview.n || 1;
264 284   host.innerHTML = "";
265 285   resp.emitters.forEach((e) => {
266 286       const d = e.det, [cls, lbl] = boxStyle(e);
267 287       const wpct = Math.max(0.8, d.bw / fs * 100);           // width = bandwidth / fs
268 288       const left = (d.fc + fs / 2) / fs * 100 - wpct / 2;    // x      = carrier on -fs/2..fs/2
269 289       const top = Math.max(0, d.t0 / n * 100);              // y      = burst start over capture
270 290       const b = document.createElement("button");
271 291       b.className = "box " + cls;
272 292       b.style.left = Math.max(0, Math.min(100 - wpct, left)) + "%";
273 293       b.style.width = wpct + "%";
274 294       b.style.top = top + "%";
275 295       b.style.height = Math.min(100 - top, Math.max(7, (d.t1 - d.t0) / n * 100)) + "%";
276 296       b.title = lbl;
277 297       b.innerHTML = `${lbl}</span>`;
278 298       b.onclick = () => drillEmitter(e);
279 299       host.appendChild(b);
280 300   });
281 301 }
282 302
283 303 function renderSurveyList(resp) {
284 304   const KIND = { linear: "디지털", fsk: "FSK", chirp: "처프/LoRa", analog: "아날로그",
285 305     tone: "톤/CW", tooshort: "짧음", error: "오류" };
286 306   const rf0 = resp.rf_center;           // real RF = capture centre + abs_fc
287 307   $("survBody").innerHTML = resp.emitters.map((e, i) => {
288 308       const [cls] = boxStyle(e), r = e.result;
289 309       const lock = r ? `

```

```

322 342   if (e.kind === "chirp" && e.info) {           // linear chirp / CSS (LoRa): characterized only
323 343       const c = e.info;
324 344       title = (c.sf ? `LoRa 계열 (SF${c.sf})` : "선형 처프 / CSS") + " - 복조 대상 아님";
325 345       // rate + direction are robust (coherent from the beat tone); bw/SF only when the
326 346       // channel is cleanly isolated enough to snap to a LoRa hypothesis.
327 347       rows = [
328 348           ["중심주파수", fmtHz(e.abs_fc)],
329 349           ["처프 방향", c.up ? "▲ 상승" : "▼ 하강"],
330 350           ["처프율", (c.mu / 1e9).toFixed(3) + " MHz/ms"]];
331 351       if (c.sf) rows.push(["대역폭", fmtHz(c.bw)], ["심볼레이트", fmtHz(c.rs)],
332 352           ["심볼시간", (c.tsym * 1e3).toFixed(2) + " ms"]);
333 353   } else {
334 354       const KIND = { analog: "아날로그 (FM/음성)", tone: "톤 / CW",
335 355           tooshort: "너무 짧은 버스트", error: "분석 오류" };
336 356       title = (KIND[e.kind] || e.kind) + " - 복조 대상 아님";
337 357       rows = [
338 358           ["중심주파수", fmtHz(e.abs_fc)], ["대역폭", fmtHz(d.bw)],
339 359           ["SNR", d.snr_db == null ? "-" : d.snr_db.toFixed(1) + " dB"],
340 360           ["심볼레이트(추정)", fmtHz(d.baud_hint)]];
341 361   }
342 362   $("survInfo").innerHTML = `<h3>${title}</h3>` +
343 363       '<div class="si-grid">' + rows.map(([k, v]) =>
344 364       `<div><div class="si-k">${k}</div><div class="si-v">${v}</div></div>`).join("") + "</div>"
345 365       ;
346 366   $("survInfo").classList.remove("hidden");
347 367   $("survInfo").scrollIntoView({ behavior: REDUCED ? "auto" : "smooth", block: "nearest" });
348 368   }
349 369   /* --- burst map: whole-record waterfall with clickable time-burst boxes (single signal) --- */
350 370   function renderBurstMap(ov, result) {
351 371       show$("burstMap");
352 372       paintWaterfall($("burstFall"), ov.waterfall, 150);
353 373       const host = $("burstBoxes"), n = ov.n || 1;
354 374       host.innerHTML = "";
355 375       (result.bursts || []).forEach((b, i) => {
356 376           const top = Math.max(0, b.start / n * 100);
357 377           const dur = (b.end - b.start) / ov.fs;
358 378           const lbl = `버스트 ${i + 1} · ${dur >= 1 ? dur.toFixed(2) + " s" : (dur * 1e3).toFixed(0)
359 379               } + " ms"`;
360 380           const box = document.createElement("button"); // full width (one signal), spans t0..t1 i
361 381               n time
362 382           box.className = "box" + (i === result.burst_idx ? " sel" : "");
363 383           box.style.left = "0%"; box.style.width = "100%"; box.style.top = top + "%";
364 384           box.style.height = Math.min(100 - top, Math.max(5, (b.end - b.start) / n * 100)) + "%";
365 385           box.title = lbl;
366 386           box.innerHTML = `<span class="box-lbl">${lbl}</span>`;
367 387           box.onclick = () => { state.burst = i; analyze(); }; // re-analyse that burst; map persist
368 388               s
369 389           host.appendChild(box);
370 390       });
371 391   }
372 392   /* --- rendering --- */
373 393   function statusOf(lock) {
374 394       if (lock >= 60) return ["복조 성공", "ok"];
375 395       if (lock >= 40) return ["부분 복조", "warn"];
376 396       return ["복조 실패", "bad"];
377 397   }
378 398   function render(d) {
379 399       hideAll();
380 400       show$("results");
381 401       $("burstMap").classList.add("hidden"); // re-shown by renderBurstMap only in multi-burst sin

```



```

    gle mode
378 398 const lock = d.quality.lock, [txt, cls] = statusOf(lock);
379 399 const pill = $("statusPill");
380 400 pill.textContent = txt;
381 401 pill.className = "pill " + cls;
382 402 $("fileName").textContent =
383 403   `${state.file.name} · ${fmtHz(d.fs)} · ${d.fmt === "real" ? "Real PCM" : "IQ"}`;
384 404 $("merVal").textContent = d.quality.mer_db == null ? "-" : d.quality.mer_db.toFixed(1) + " dB"
    ;
385 405 $("evmVal").textContent = d.quality.evm == null ? "-" : (d.quality.evm * 100).toFixed(1) + " %"
    ;
386 406 $("snrVal").textContent = snrText(d);
387 407
388 408 const flags = [];
389 409 if (d.family === "fsk") flags.push('<span class="chip warn">FSK 계열 · 주파수 판별</span>');
390 410 if (d.eq && d.eq.applied) flags.push('<span class="chip hit">등화기 적용 · ${
391 411   d.eq.mode === "fse" ? "T/2 분수간격" : "심볼간격"></span>');
392 412 if (d.detected.alias_resolved) flags.push('<span class="chip warn">반송파 앨리어스 보정</span>
    ');
393 413 if (d.detected.baud_fallback) flags.push('<span class="chip warn">심볼레이트 대역폭 폴백</span>
    ');
394 414 if (d.detected.carrier_ambiguous) flags.push('<span class="chip warn">반송파 모호 · 앨리어
    싱 가능</span>');
395 415 $("flagRow").innerHTML = flags.join("");
396 416
397 417 renderBursts(d);
398 418 animateGauge(lock, cls);
399 419 renderParams(d);
400 420 drawSpectrum(d);
401 421 drawWaterfall(d);
402 422 startPlay(d);
403 423 }
404 424 function snrText(d) {
405 425   const s = d.snr_est_db;
406 426   if (s == null || d.quality.lock < 30) return "-";
407 427   return (s > 28 ? "≥28" : s.toFixed(1)) + " dB";
408 428 }
409 429
410 430 function renderBursts(d) {
411 431   const row = $("burstRow"), bs = d.bursts || [];
412 432   // a survey-drilled emitter is already a cut channel: burst re-selection would
413 433   // re-post the whole capture, so it is disabled while drilled. When the burst MAP
414 434   // is shown (multi-burst single signal) the map replaces the chips.
415 435   if (state.drilled || state.overview || bs.length < 2) { row.innerHTML = ""; return; }
416 436   const dur = (b) => {
417 437     const sec = (b.end - b.start) / d.fs;
418 438     return sec >= 1 ? sec.toFixed(2) + " s" : (sec * 1e3).toFixed(1) + " ms";
419 439   };
420 440   row.innerHTML = bs.map((b, i) =>
421 441     `<button class="chip-btn${i === d.burst_idx ? " on" : ""}" data-i="${i}">` +
422 442     `버스트 ${i + 1} · ${dur(b)}</button>`).join("");
423 443   row.querySelectorAll("button").forEach((el) => {
424 444     el.onclick = () => { state.burst = +el.dataset.i; analyze(); };
425 445   });
426 446 }
427 447
428 448 function chip(hit, truthTxt) {
429 449   return `<span class="chip ${hit ? "hit" : "miss"}">정답 ${truthTxt} · ${hit ? "일치" : "불일치"
    }</span>`;
430 450 }

```

```

431 451 function renderParams(d) {
432 452   const det = d.detected, t = state.sidecar && state.sidecar.truth;
433 453   const near = (a, b, tol) => Math.abs(a - b) <= tol;
434 454   const rows = [
435 455     ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc),
436 456       t && t.fc != null && chip(near(det.fc, t.fc, Math.max(300, 0.02 * t.baud)), fmtHz(t.fc)),
437 457       det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
438 458     ["심볼레이트", fmtHz(det.baud), t && t.baud != null && chip(near(det.baud, t.baud, 0.03 * t.
439 459       baud), fmtHz(t.baud)),
440 460       det.baud_conf ? `스펙트럴 라인 강도 ×${Math.round(det.baud_conf)}` : ""],
441 461     ["변조방식", det.mod.toUpperCase(), t && t.mod != null && chip(det.mod === t.mod, t.mod.toUp
442 462       perCase()),
443 463       det.symmetry ? `대칭 ${det.symmetry}` : "주파수 레벨"],
444 464   ];
445 465   if (d.family === "fsk") {
446 466     rows.push(["변조지수 h", det.h.toFixed(2),
447 467       t && t.h != null && chip(near(det.h, t.h, 0.15), t.h.toFixed(2)), ""]);
448 468   } else {
449 469     rows.push(["롤오프", det.rolloff.toFixed(2),
450 470       t && t.rolloff != null && chip(near(det.rolloff, t.rolloff, 0.11), t.rolloff.toFixed(2))
451 471       , ""]);
452 472   }
453 473   $("paramGrid").innerHTML = rows.map(([label, val, hit, note]) => `
454 474     <div class="card param">
455 475       <div class="param-label">${label}</div>
456 476       <div class="param-val">${val}</div>
457 477       ${note ? `<div class="param-note">${note}</div>` : ""}
458 478       ${hit ? `<div class="chips">${hit}</div>` : ""}
459 479     </div>`).join("");
460 480
461 481 function animateGauge(lock, cls) {
462 482   const arc = $("donutArc"), num = $("lockNum"), C = 2 * Math.PI * 52;
463 483   arc.style.stroke = { ok: "#12b886", warn: "#f59f00", bad: "#fa5252" }[cls];
464 484   arc.style.strokeDasharray = C;
465 485   const target = C * (1 - Math.max(0, Math.min(100, lock)) / 100);
466 486   if (REDUCED) { arc.style.strokeDashoffset = target; num.textContent = Math.round(lock); return
467 487     ; }
468 488   arc.style.strokeDashoffset = C;
469 489   requestAnimationFrame(() => { arc.style.strokeDashoffset = target; });
470 490   const t0 = performance.now();
471 491   const step = (t) => {
472 492     const k = Math.min(1, (t - t0) / 900);
473 493     num.textContent = Math.round(lock * (1 - Math.pow(1 - k, 3)));
474 494     if (k < 1) requestAnimationFrame(step);
475 495   };
476 496   requestAnimationFrame(step);
477 497
478 498   /* --- canvas helpers --- */
479 499   function fit(cv, hCss) {
480 500     const dpr = window.devicePixelRatio || 1, w = cv.clientWidth || 320;
481 501     const h = hCss || w;
482 502     cv.width = w * dpr; cv.height = h * dpr;
483 503     const g = cv.getContext("2d");
484 504     g.setTransform(dpr, 0, 0, dpr, 0, 0);
485 505     return [g, w, h];
486 506   }
487 507   function pct(a, p) {
488 508     const s = [...a].sort((x, y) => x - y);

```

```

487 507     return s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))];
488 508 }
489 509
490 510 /* --- spectrum --- */
491 511 function drawSpectrum(d) {
492 512     const [g, w, h] = fit($("#specCanvas"), 120);
493 513     g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, h);
494 514     if (!d.spectrum) return;
495 515     const f = d.spectrum.f, db = d.spectrum.db;
496 516     const lo = pct(db, 0.02), hi = pct(db, 1) + 2;
497 517     const fmin = f[0], fmax = f[f.length - 1];
498 518     const X = (v) => (v - fmin) / (fmax - fmin) * w;
499 519     const Y = (v) => h - (Math.max(lo, Math.min(hi, v)) - lo) / (hi - lo) * (h - 8) - 4;
500 520     const det = d.detected, half = det.baud / 2e3;
501 521     g.strokeStyle = "rgba(120,180,255,.35)"; g.setLineDash([4, 4]);
502 522     for (const gf of [det.fc / 1e3 - half, det.fc / 1e3 + half]) {
503 523         g.beginPath(); g.moveTo(X(gf), 0); g.lineTo(X(gf), h); g.stroke();
504 524     }
505 525     g.setLineDash([]);
506 526     g.strokeStyle = "rgba(255,255,255,.45)";
507 527     g.beginPath(); g.moveTo(X(det.fc / 1e3), 0); g.lineTo(X(det.fc / 1e3), h); g.stroke();
508 528     g.strokeStyle = "#5aa9ff"; g.lineWidth = 1.4; g.beginPath();
509 529     f.forEach((v, k) => (k ? g.lineTo(X(v), Y(db[k])) : g.moveTo(X(v), Y(db[k]))));
510 530     g.stroke();
511 531     g.fillStyle = "rgba(255,255,255,.5)"; g.font = "10px sans-serif";
512 532     g.fillText(sig(fmin) + " kHz", 4, h - 4);
513 533     const tmax = sig(fmax) + " kHz";
514 534     g.fillText(tmax, w - g.measureText(tmax).width - 4, h - 4);
515 535 }
516 536
517 537 /* --- waterfall --- */
518 538 function drawWaterfall(d) { paintWaterfall($("#fallCanvas"), d.waterfall, 150); }
519 539 function paintWaterfall(canvas, wf, hCss) {
520 540     const [g, w, h] = fit(canvas, hCss);
521 541     g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, h);
522 542     if (!wf || !wf.rows) return;
523 543     const lo = pct(wf.db, 0.05), hi = pct(wf.db, 0.95);
524 544     const img = g.createImageData(wf.bins, wf.rows);
525 545     for (let k = 0; k < wf.db.length; k++) {
526 546         const t = Math.max(0, Math.min(1, (wf.db[k] - lo) / (hi - lo + 1e-9)));
527 547         const o = k * 4; // dark -> blue -> white ramp
528 548         img.data[o] = t < 0.5 ? 13 + t * 120 : 73 + (t - 0.5) * 364;
529 549         img.data[o + 1] = t < 0.5 ? 19 + t * 260 : 149 + (t - 0.5) * 212;
530 550         img.data[o + 2] = t < 0.5 ? 32 + t * 446 : 255;
531 551         img.data[o + 3] = 255;
532 552     }
533 553     createImageBitmap(img).then((bmp) => { g.imageSmoothingEnabled = true; g.drawImage(bmp, 0, 0
534 554         , w, h); });
535 555 }
536 556
537 557 /* --- constellation playback (the headline) --- */
538 558 const play = { raf: 0, i: 0, on: false, data: null };
539 559 let CG = null; // cached constellation geometry
540 560
541 561 function stopPlay() { cancelAnimationFrame(play.raf); play.raf = 0; play.on = false; }
542 562 function setPlayBtn(on) { $("#playBtn").textContent = on ? " 일시정지 " : "▶ 재생 "; }
543 563
544 564 function startPlay(d) {
545 565     stopPlay();
546     play.data = d;

```

```

546 566   play.i = 0;
547 567   const n = d.constellation.i.length;
548 568   $("scrub").max = String(Math.max(1, n));
549 569   $("playInfo").textContent = `${n.toLocaleString()} 심볼 · 시간 순서로 재생 (잔광 효과)`;
550 570   drawConstBase(d);
551 571   if (REDUCED) {                                     // static full view, no motion
552 572       drawPoints(d, 0, n);
553 573       $("scrub").value = String(n);
554 574       setPlayBtn(false);
555 575       return;
556 576   }
557 577   play.on = true;
558 578   setPlayBtn(true);
559 579   play.raf = requestAnimationFrame(loop);
560 580 }
561 581 $("playBtn").onclick = () => {
562 582   if (!play.data) return;
563 583   if (play.on) { stopPlay(); setPlayBtn(false); }
564 584   else { play.on = true; setPlayBtn(true); play.raf = requestAnimationFrame(loop); }
565 585 };
566 586 $("scrub").oninput = () => {
567 587   if (!play.data) return;
568 588   stopPlay(); setPlayBtn(false);
569 589   play.i = +$("scrub").value;
570 590   drawConstBase(play.data);
571 591   drawPoints(play.data, 0, play.i);    // redraw history up to the scrub point
572 592 };
573 593
574 594 function loop() {
575 595   const d = play.data, n = d.constellation.i.length;
576 596   const step = Math.max(1, Math.round(n / 600) * (+$("speedSel").value));
577 597   const to = Math.min(n, play.i + step);
578 598   fade();                                     // phosphor persistence
579 599   drawPoints(d, play.i, to);
580 600   play.i = to >= n ? 0 : to;
581 601   if (play.i === 0) drawConstBase(d); // loop: clear the trail
582 602   $("scrub").value = String(play.i);
583 603   if (play.on) play.raf = requestAnimationFrame(loop);
584 604 }
585 605
586 606 function drawConstBase(d) {
587 607   const [g, w] = fit($("constCanvas"));
588 608   const fsk = d.family === "fsk";
589 609   const I = d.constellation.i, Q = d.constellation.q;
590 610   const ideal = fsk ? [] : idealPoints(d.detected.mod);
591 611   let lim = 0;
592 612   for (let k = 0; k < I.length; k++) lim = Math.max(lim, Math.abs(I[k]), Math.abs(Q[k]));
593 613   for (const p of ideal) lim = Math.max(lim, Math.abs(p.i), Math.abs(p.q));
594 614   lim = (lim || 1) * 1.12;
595 615   const R = w / 2 * 0.9;
596 616   CG = { g, w, lim, R, map: (re, im) => [w / 2 + re / lim * R, w / 2 - im / lim * R] };
597 617
598 618   g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, w);
599 619   g.strokeStyle = "rgba(255,255,255,.06)"; g.lineWidth = 1;
600 620   for (let f = -1; f <= 1; f += 0.5) {
601 621       const [gx] = CG.map(f, 0), [, gy] = CG.map(0, f);
602 622       g.beginPath(); g.moveTo(gx, 0); g.lineTo(gx, w); g.stroke();
603 623       g.beginPath(); g.moveTo(0, gy); g.lineTo(w, gy); g.stroke();
604 624   }
605 625   if (fsk) {                                     // frequency levels as dashed bands

```

```

606 626     g.strokeStyle = "rgba(255,255,255,.5)"; g.setLineDash([6, 5]);
607 627     for (const lv of [...new Set(I.map((v) => Math.round(v)))]).filter((v) => Math.abs(v) <= 8)
        L L
608 628         const [, yy] = CG.map(0, lv);
609 629         g.beginPath(); g.moveTo(0, yy); g.lineTo(w, yy); g.stroke();
610 630     }
611 631     g.setLineDash([]);
612 632 } else {
613 633     g.strokeStyle = "rgba(255,255,255,.75)"; g.lineWidth = 1.4;
614 634     for (const p of ideal) {
615 635         const [x, y] = CG.map(p.i, p.q);
616 636         g.beginPath(); g.arc(x, y, 6, 0, 7); g.stroke();
617 637     }
618 638 }
619 639 }
620 640 function fade() {
621 641     if (!CG) return;
622 642     CG.g.fillStyle = "rgba(13,19,32,.14)"; // translucent wash = phosphor decay
623 643     CG.g.fillRect(0, 0, CG.w, CG.w);
624 644 }
625 645 function drawPoints(d, from, to) {
626 646     if (!CG || to <= from) return;
627 647     const g = CG.g, I = d.constellation.i, Q = d.constellation.q;
628 648     const fsk = d.family === "fsk";
629 649     const a = Math.max(0.08, Math.min(0.55, 150 / (to - from)));
630 650     g.globalCompositeOperation = "lighter";
631 651     g.fillStyle = `rgba(90,170,255,${a})`;
632 652     for (let k = from; k < to; k++) {
633 653         // FSK has no I/Q plane: spread levels vertically, jitter horizontally for density
634 654         const [x, y] = fsk ? CG.map((k % 89) / 89 - 0.5, I[k]) : CG.map(I[k], Q[k]);
635 655         g.beginPath(); g.arc(x, y, 1.8, 0, 7); g.fill();
636 656     }
637 657     g.globalCompositeOperation = "source-over";
638 658 }
639 659
640 660 /* --- downloads --- */
641 661 function saveBlob(name, text, type) {
642 662     const url = URL.createObjectURL(new Blob([text], { type }));
643 663     const a = document.createElement("a");
644 664     a.href = url; a.download = name; a.click();
645 665     URL.revokeObjectURL(url);
646 666 }
647 667 const stem = () => state.file.name.replace(/\.[^.]*/, "");
648 668 $("copyBits").onclick = async (e) => { // decoded bitstream -> clipboard, with brief feedback
649 669     const btn = e.currentTarget, was = btn.textContent;
650 670     try { await navigator.clipboard.writeText(state.resp.bits); btn.textContent = "복사됨 ✓"; }
651 671     catch { btn.textContent = "복사 실패"; }
652 672     setTimeout(() => { btn.textContent = was; }, 1400);
653 673 };
654 674 $("dlBits").onclick = () => saveBlob(stem() + ".bits.txt", state.resp.bits, "text/plain");
655 675 $("dlSyms").onclick = () => {
656 676     const c = state.resp.constellation;
657 677     saveBlob(stem() + ".symbols.csv", "i,q\n" + c.i.map((v, k) => `${v},${c.q[k]}`).join("\n"), "t
        L L
        ext/csv");
658 678 };
659 679 $("dlReport").onclick = () =>
660 680     saveBlob(stem() + ".report.json", JSON.stringify(state.resp, null, 2), "application/json");
661 681
662 682 /* --- view helpers --- */
663 683 function show(el) { el.classList.remove("hidden"); }

```

```
664 684 function hideAll() {
665 685   stopPlay();
666 686   ["metaForm", "loading", "errorCard", "results", "batchCard", "surveyCard"]
667 687   .forEach((id) => $(id).classList.add("hidden"));
668 688   $("backBatch").classList.add("hidden");
669 689 }
670 690 function showError(msg) { hideAll(); $("errMsg").textContent = msg; show($("errorCard")); }
671 691
672 692 /* --- dropzone wiring --- */
673 693 const drop = $("dropCard"), input = $("fileInput");
674 694 // reset value first: re-picking the SAME file fires no change event otherwise
675 695 const pick = () => { input.value = ""; input.click(); };
676 696 drop.onclick = pick;
677 697 drop.onkeydown = (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault(); pick()
678 698   ; } };
679 699 input.onchange = () => { if (input.files.length) acceptFiles(input.files); };
680 700 drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("drag"); });
681 701 drop.addEventListener("dragleave", () => drop.classList.remove("drag"));
682 702 drop.addEventListener("drop", (e) => {
683 703   e.preventDefault(); drop.classList.remove("drag");
684 704   if (e.dataTransfer.files.length) acceptFiles(e.dataTransfer.files);
685 705 });
```